# The ZataOne TextExtractor: Code and Structure

*A technical walkthrough of* `src/zataone/extractors/text_extractor.py` *(v2.0)*

*ZataOne Engineering — July 2026*

---

**Abstract.** The TextExtractor is the first sensor in the ZataOne compliance pipeline for text assets. It is deliberately *not* a machine-learning model: eight of its eleven signal families are deterministic keyword and regular-expression detectors, and the remaining three are lightweight, optional NLP add-ons (lexicon sentiment, n-gram language identification, and a closed-form readability formula). This document explains where the extractor sits in the architecture, walks through each section of the source file, documents every signal type it emits, and closes with the design principles and known limitations that shape it.

---

## Contents

# 1 Role in the pipeline

ZataOne separates *observation* from *judgement*. Extractors observe: they turn raw content into structured **signals** (facts with a type, a confidence, and a location). The policy engine judges: it evaluates YAML-defined rules over those signals and over the unified document text. The TextExtractor is the observation layer for assets of type `text` — ad copy submitted directly as a string. Text recovered from images travels a different road (the OCR extractor), and audio is transcribed by Whisper before any text analysis.

Within one compliance check the extractor is invoked by `extract_signals_parallel()` alongside whichever other extractors the asset type activates. Its output feeds three consumers: the **DocumentBuilder** (which assembles the normalized document), the **PolicyEngine** (whose keyword rules fire against signal values), and the **persistence layer** (each signal becomes a row in the `signals` table, linkable as evidence).

```
POST /assets {content, type: "text"}
        |
        v
CompliancePipeline.run()
  -> select_extractors_for_asset()      # text asset -> TextExtractor
  -> extract_signals_parallel()
       -> TextExtractor.extract(asset)  # THIS FILE
            returns [Signal, Signal, ...]
  -> DocumentBuilder.build(asset, signals)
  -> PolicyEngine.evaluate(signals, document)
  -> Evidence -> Verdict -> persist
```

Figure 1: Where TextExtractor.extract() runs inside one compliance check.

# 2 File structure at a glance

The source file is 464 lines and reads top-to-bottom in seven blocks. Everything above the class definition is module-level configuration — compiled once at import, shared by every call.

| Lines | Block | Purpose |
|---|---|---|
| 1–32 | Docstring + imports | Lists all 11 signal types; declares the extractor is pure (no DB writes) |
| 35–58 | HIGH_RISK_KEYWORDS | 20 literal strings checked by substring match |
| 61–64 | Claim patterns | Two compiled regexes: percentages and time promises |
| 67–151 | Entity + toxicity regexes | Four verbose regex blocks: monetary, contact, medical, legal; one deceptive-pattern set |
| 154–196 | Library loaders | Cached availability checks for nltk/VADER and langdetect |
| 199–237 | Readability | Syllable counter + Flesch reading-ease, pure Python |
| 240–252 | Signal dataclass | The record every detector emits |
| 255–464 | TextExtractor class | extract() orchestration + one helper per NLP signal |

Table 1: Layout of text_extractor.py.

The class inherits from `BaseExtractor` and identifies itself as `extractor_id = "text_extractor"`, version 2.0. Registration happens in `CompliancePipeline._load_domain_extractors()`; per-asset activation is decided by `core/extractor_plan.py`, which always allows the text extractor for text and audio assets.

# 3 The signal taxonomy

Every detector emits the same record shape; only `signal_type`, `confidence`, and the payload differ. Confidences are fixed per category and encode how unambiguous that detector is, not a model probability: an exact keyword match cannot be wrong about the keyword being present (1.0), while a legal-sounding phrase is more contextual (0.85).

| Signal type | Method | Example hit | Conf. |
|---|---|---|---|
| `keyword` | substring over 20 terms | “guaranteed”, “cure” | 1.00 |
| `percentage_claim` | regex \d+% | “99%” | 1.00 |
| `time_claim` | regex number + unit | “30 days” | 1.00 |
| `entity_monetary` | regex | “$1,234.56”, “FREE” | 0.95 |
| `entity_contact` | regex | email, URL, phone | 0.90 |
| `entity_medical` | regex vocabulary | “FDA-approved”, “detox” | 0.85 |
| `entity_legal` | regex vocabulary | “no refund”, “arbitration” | 0.85 |
| `toxicity` | regex trope list | “act now”, “doctors hate” | 0.80 |
| `sentiment` | VADER lexicon (opt.) | positive / negative / neutral | \|compound\| |
| `language` | langdetect n-grams (opt.) | “en” @ 0.99 | detector prob. |
| `readability` | Flesch formula | score 62.4, “standard” | 1.00 |

Table 2: The eleven signal families. The last three run only when len(text) ≥ 20.

The first eight rows are **rule fuel**: the policy engine’s prohibited-term and pattern rules match against them (and against the unified document text). The last three are **advisory context**: nothing in the current YAML rules fires on sentiment, language, or readability — they enrich the evidence graph and the LLM review context. This distinction matters for failure analysis: losing an advisory signal cannot change a verdict, which is why their libraries are allowed to be optional.

# 4 Code walkthrough

## 4.1 Keyword lists and claim patterns

The keyword list is a flat Python list of 20 lower-case strings spanning three risk clusters: absolute guarantees (“guaranteed”, “100%”, “risk-free”), health hype (“cure”, “scientifically proven”, “lose weight fast”), and income schemes (“get rich quick”, “unlimited income”). Matching is a plain `in` test against the lower-cased content — O(n) per keyword, no word boundaries, so “cure” also matches “secure” (a known trade-off, discussed in §6).

```
HIGH_RISK_KEYWORDS = [
    "guaranteed", "instant", "100%", "cure", "permanent",
    "no risk", "risk-free", "scientifically proven", ...
]

PERCENTAGE_PATTERN = re.compile(r"\d+%")
TIME_CLAIM_PATTERN = re.compile(
    r"\d+\s*(days?|hours?|minutes?|weeks?|months?)", re.IGNORECASE)
```

Listing 1: Keywords and the two numeric-claim regexes (lines 37–64).

The two claim patterns capture the numeric skeletons of misleading advertising: a bare percentage ("99% effective") and a time-bound promise ("results in 30 days"). They intentionally over-collect — "30 days" in "return within 30 days" also fires — because context and exceptions are the *policy engine's* job, not the extractor's.

### 4.2 Entity patterns (monetary, contact, medical, legal)

Four verbose-mode regexes (`re.VERBOSE` permits comments and whitespace inside the pattern) detect entity families. Each alternation branch is a shape of the entity:

```
_MONETARY_RE = re.compile(r"""(?:
    \$\s*[\d,]+(?:\.\d{1,2})?                    # $1,234.56
    | [\d,]+(?:\.\d{1,2})?\s*(?:USD|EUR|GBP)\b   # 1234 USD
    | \bfree\b | \bFREE\b                         # "free" claims
    | no[\s-]+charge | no[\s-]+cost
)""", re.VERBOSE | re.IGNORECASE)
```

Listing 2: The monetary entity regex (lines 69–77), one branch per entity shape.

**Contact** covers emails, URLs, and North-American phone formats — relevant because several platform policies restrict where contact details may appear in creatives. **Medical** is a vocabulary of claim verbs and health tokens (treat/cure/heal/diagnose, "FDA-approved", "clinical trial", "weight-loss", "immune boost") with morphological suffixes folded into the pattern (`treat(?:ment|s|ed|ing)?`). **Legal** collects warranty, liability, arbitration, and refund language — signals that disclaimers exist (or that consumer rights are being waived).

### 4.3 The "toxicity" pattern set

Despite the name, this is *not* an ML toxicity classifier. It is a curated regex list of manipulative-advertising tropes, grouped by tactic: false urgency ("limited time offer", "act now", "while supplies last"), conspiratorial framing ("doctors hate", "they don't want you to know"), miracle claims ("miracle cure/pill/formula"), and quantified bait ("lose 20 pounds", "earn $500 per day"). Confidence is the lowest of the deterministic families (0.80) because these phrases occasionally appear in legitimate copy.

### 4.4 Optional-library loaders and graceful degradation

Sentiment (NLTK/VADER) and language identification (langdetect) are optional dependencies. Each has a module-level cache dict and a checker function that attempts the import exactly once per process, downloads the VADER lexicon on first use if NLTK is present, logs an actionable warning on failure (including the pip command to fix it), and remembers the answer:

```
_vader_state = {"checked": False, "available": False}

def _vader_available() -> bool:
    if _vader_state["checked"]:
        return _vader_state["available"]        # cached
    _vader_state["checked"] = True
    try:
        import nltk
        nltk.data.find("sentiment/vader_lexicon.zip")  # or download
        from nltk.sentiment.vader import SentimentIntensityAnalyzer
        _vader_state["available"] = True
    except Exception as exc:
        logger.warning("VADER unavailable; ... Fix: pip install nltk")
        _vader_state["available"] = False
    return _vader_state["available"]
```

Listing 3: The availability-check pattern (lines 160–181; langdetect is analogous).

This is the *acceptable* form of silent degradation: because sentiment and language are advisory-only signals, skipping them can never flip a verdict toward approval. Contrast the vision extractors, where a silent model-load failure could suppress a weapons detection — which is why the pipeline now caps verdicts at REVIEW_REQUIRED when a planned extractor fails (the fail-to-review rule).

### 4.5 Readability: pure-Python Flesch reading ease

Readability needs no library at all. Syllables are approximated by counting vowel groups (with a silent-e correction and a floor of one); sentences are split on terminal punctuation. The classic Flesch formula is then applied and clamped to [0, 100]:

```
score = 206.835 - 1.015 * (words / sentences)
                - 84.6  * (syllables / words)

# >= 70 easy | >= 50 standard | >= 30 difficult | else very_difficult
```

Listing 4: The Flesch reading-ease computation (lines 215–237).

Compliance relevance: several regulators treat illegible or needlessly complex disclosure text as a dark pattern. A “very_difficult” score on a disclaimer-heavy creative is a useful review cue even though no rule currently fires on it.

### 4.6 The Signal dataclass and the extract() flow

Every detector routes through one factory, `_make()`, which stamps a fresh UUID, the extractor id as `source_model`, and the category’s confidence:

```
@dataclass
class Signal:
    signal_id: str          # uuid4
    signal_type: str        # e.g. "keyword"
    source_model: str       # "text_extractor"
    confidence: float       # fixed per category
    raw_data: dict          # {type, text, value?, offset?}
    bounding_box: None = None   # text has offsets, not boxes
```

Listing 5: The signal record (lines 243–252).

`extract()` itself is a straight-line orchestration, numbered 1–11 in the source. It guards on asset type (non-text assets return an empty list), lower-cases once, then runs the eight deterministic detectors in order; every regex hit records `offset: match.start()` so the evidence chain can point at the exact character position in the ad copy. If the text is at least 20 characters (`_NLP_MIN_LENGTH`), the three NLP signals are appended — sentiment and language returning `None` harmlessly when their libraries are missing, readability always present. There is no early exit and no shared state: the function is pure, and identical input yields byte-identical output apart from the generated UUIDs.

## 5 Design principles

**Sensor, not judge.** Terms such as “guaranteed” appear both here and in the policy YAML (`meta_ads.yaml`). That duplication is intentional layering, not redundancy: the extractor asserts *“the word is present at offset 12”*; the rule decides *“in this context, with no exception phrase nearby, that violates misleading_exaggerated_claims at severity HIGH”*. Exceptions (“quit smoking”), context requirements (medical terms must co-occur), and severities live exclusively in YAML — so policy can change without touching extractor code.

**Determinism as a legal property.** The eight core detectors involve no randomness, no network, no model weights. A verdict challenged a year later can be reproduced exactly from the stored text and the policy pack content hash. This is the foundation of ZataOne’s “legally defensible decisions” claim, and it is why the ML-flavoured signals are quarantined into the advisory tier.

**Offsets are evidence.** Persisting `match.start()` looks trivial, but it is what makes the explainability graph clickable: verdict → violation → evidence → signal → character range in the original copy.

**Fixed confidences over fake probabilities.** Rather than pretending a regex has a probabilistic confidence, each category carries a hand-set constant that feeds the verdict scorer’s confidence multiplier (>0.9 → ×1.0; 0.7–0.9 → ×0.7; <0.7 → ×0.3). The numbers are policy-tunable and honest about their origin.

## 6 Limitations and how the rest of the system compensates

| Limitation | Consequence | Compensating layer |
|---|---|---|
| `Substring matching, no word boundaries` | “cure” matches “secure” (false positive) | Policy-engine context terms and exceptions; human review labels the false positive, feeding rule tuning |
| `Spelling-exact` | “gauranteed” evades detection | SigLIP embedding similarity; Gemini advisory read; review-loop labels reveal evasions |
| `English-only vocabularies` | Non-English copy under-detected | language signal records the gap; multilingual packs are roadmap work |
| `No paraphrase understanding` | “your money back, promise!” ≠ “money-back guarantee” | LLM advisory second read flags divergence from the deterministic verdict |
| `Keyword list is hand-maintained` | Coverage drifts as ad language evolves | review_decisions.violation_feedback accumulates labeled data for systematic list updates |

Table 3: Known limitations and the architectural answer to each.

None of these weaknesses is hidden: they are the accepted cost of a decision layer that is fully auditable. The system’s posture is that probabilistic components (embeddings, LLMs) may *widen the net* and *advise*, but only deterministic evidence — or a recorded human decision — may set the final verdict.

---

Source reference: src/zataone/extractors/text_extractor.py (464 lines, v2.0) · related: core/extractor_plan.py, document/builder.py, policy_engine/engine.py, services/verdict_service.py